

```
<style>.fase-header { display: flex; align-items: center; width: 100%; margin: 10px 0; padding: 40px; background-color: rgba(255, 255, 255, 0.1); font-size: 25px; } </style>
```

Plan de trabajo - Ecommerce Template IA

Objetivo

Construir un template de e-commerce modular y reutilizable, desarrollado con ayuda de agentes de IA y modelos open source.

FASE 0: Definición

0.1. Definir alcance del MVP

Definición del alcance:

1. ¿Puede un usuario comprar un producto?
2. ¿La arquitectura es reutilizable para otros clientes?

Alcance del MVP:

▷ Funcionalidades incluidas:

- Catálogo
- Mostrar listado de productos.
- Mostrar detalle de un producto.
- Filtrar productos por categoría.
- Buscar productos por nombre.

▷ Carrito:

- Agregar productos.
- Modificar cantidad.
- Eliminar productos.
- Ver subtotal.

▷ Checkout:

- Resumen del pedido.
- Captura de datos del cliente.
- Confirmación del pedido.

* No incluir pasarela de pago real en el MVP. Podemos simular el pago o dejar preparada una interfaz para integrarla más adelante.

▷ Usuarios:

- Registro.
- Inicio de sesión.
- Cierre de sesión.
- Perfil básico.

▷ Panel administrativo básico:

* No hace falta construir un CMS completo.

Solo lo suficiente para:

- CRUD de productos.
- CRUD de categorías.
- Ver pedidos.

▷ Backend:

- API REST.
- Autenticación.
- Gestión de productos.
- Gestión de pedidos.
- Gestión de usuarios.

▷ Base de datos:

+ Modelos iniciales:

- Users
- Products
- Categories
- Orders
- OrderItems

▷ Frontend

+ Páginas:

- Home
- Catálogo
- Producto
- Carrito
- Checkout
- Login
- Registro
- Perfil

▷ Diseño:

- Responsive.
- Modo claro.
- Sistema de componentes reutilizable.

* No hace falta un modo oscuro para el MVP.

▷ Template:

El proyecto debe poder cambiar fácilmente:

- Logo.
- Colores.
- Tipografía.
- Nombre de la tienda.
- Productos de ejemplo.

* Sin modificar el código de negocio.

Queda fuera del MVP:

Funcionalidades explícitamente fuera del MVP

* Esto es igual de importante que lo anterior.

No incluye todavía:

- ✗ Cupones de descuento.
- ✗ Wishlist.
- ✗ Reviews.
- ✗ Variantes complejas (talla, color, etc.).
- ✗ Multiidioma.
- ✗ Multimoneda.
- ✗ Integración con Stripe, PayPal, Mercado Pago, etc.

- ✗ Emails transaccionales.
- ✗ Gestión avanzada de inventario.
- ✗ Dashboard con estadísticas.
- ✗ SEO avanzado.
- ✗ Analytics.
- ✗ Multi-vendedor.
- ✗ Multi-tienda.

**** Criterio para dar el MVP por terminado ****

El MVP estará completo cuando sea posible:

- Configurar una nueva tienda cambiando solo archivos de configuración.
- Crear productos desde el panel administrativo.
- Navegar por el catálogo.
- Agregar productos al carrito.
- Simular una compra.
- Registrar usuarios.
- Consultar los pedidos generados.

0.2. Elegir stack

Frontend

- Astro
- React

Estilos

- Tailwind CSS

ORM

- Drizzle

Base de datos

- PostgreSQL/SQLite

**** Definir convenciones y estructura del repositorio.****

0.3. Definiciones

Separación de apps ✓

División del sistema en aplicaciones independientes (web, admin, api) donde cada una tiene un propósito concreto y ejecutable.

Separación de packages ✓

División del código reutilizable en módulos independientes (ui, db, auth, etc.) que no dependen de una app específica.

Sistema de temas ✓

Estructura de configuración que define la identidad visual de la tienda (colores, tipografía, branding) sin tocar la lógica del sistema.

Sistema de UI ✓

Conjunto de componentes reutilizables y agnósticos del negocio que construyen toda la interfaz visual del sistema.

Sistema de documentación ✓

Conjunto de archivos que explican la arquitectura, decisiones y reglas del proyecto para que pueda entenderse y mantenerse sin depender del código.

Flujo del sistema ✓

Definición de cómo circula la información entre módulos:

```
admin → crea datos → api los procesa → web los muestra → usuario interactúa  
→ api guarda cambios
```

FASE 1: Arquitectura

1.1. Estructura de carpetas:

Carpeta	Propósito
<code>.github/</code>	GitHub Actions, plantillas de issues, PR, etc.
<code>agents/</code>	Definición de los agentes (rol, responsabilidades, reglas).
<code>apps/</code>	Aplicaciones (<code>web</code> , <code>admin</code> , <code>api</code>).
<code>docs/</code>	Documentación del proyecto.
<code>packages/</code>	Código reutilizable entre aplicaciones.

Carpeta	Propósito
<code>prompts/</code>	Prompts que utilizan los agentes.
<code>scripts/</code>	Scripts de automatización.
<code>templates/</code>	Datos o configuraciones base para nuevas tiendas.
<code>themes/</code>	Temas visuales (colores, tipografías, branding).

1.2. Diseño modular

`apps/`

1. `apps/web` (tienda pública)

Es la aplicación principal.

Qué hace:

- Mostrar productos
- Catálogo
- Detalle de producto
- Carrito
- Checkout
- Login / registro

Idea clave: Es la experiencia del cliente final comprando.

2. `apps/api` (backend)

Es el cerebro del sistema.

Qué hace:

- Productos (CRUD básico)
- Usuarios
- Carrito / sesiones
- Pedidos
- Autenticación

Idea clave: Aquí vive toda la lógica de negocio.

3. `apps/admin` (panel interno)

Versión simple al inicio.

Qué hace:

- Crear productos

- Ver productos
- Ver pedidos

Idea clave: Herramienta mínima para poder alimentar el sistema sin tocar código.

Flujo:

```
admin → crea producto
api → guarda producto
web → muestra producto
usuario → compra producto
api → crea pedido
admin → ve pedido
```

packages/

1. packages/db (base de datos)

Qué contiene:

- esquema de Drizzle
- migraciones
- conexiones a DB

Para qué sirve:

- Definir toda la estructura de datos del ecommerce.

Ejemplo:

```
- users
- products
- orders
- order_items
- sessions
```

2. packages/ecommerce-core (lógica de negocio)

Qué contiene:

- lógica de carrito
- lógica de pedidos
- reglas de negocio (ej: stock, precios)

Para qué sirve:

- Evitar que la lógica viva en frontend o backend directamente.

Ejemplo:

```
- addToCart()  
- calculateTotal()  
- createOrder()
```

3. packages/ui (design system)

Qué contiene:

- botones
- inputs
- cards
- modales
- layout components

Para qué sirve:

- Reutilizar UI entre web y admin.

4. packages/config (configuración del template)

Qué contiene:

- configuración del cliente
- branding
- flags del sistema

Ejemplo:

```
{  
  storeName: "Mi tienda",  
  currency: "EUR",  
  theme: "default"  
}
```

5. packages/auth (autenticación)

Qué contiene:

- login logic
- registro
- sesiones
- helpers de auth
-

Para qué sirve:

- Evitar duplicar auth en web/admin/api.

6. packages/utils (helpers genéricos)

Qué contiene:

- formateo de precios
 - validaciones
 - helpers de fecha
 - funciones puras
-

1.3. Sistema de temas

themes/

- themes/default.ts
- themes/minimal.ts
- themes/modern.ts

Cada archivo es un tema.

Alcance del tema (mínimo viable)

1. Colores

- primary
- secondary
- background
- text

2. Tipografía

- font sans
- font heading

3. UI Básica

- border radius
- spacing scale (opcional en MVP)

4. Branding

- nombre tienda
- logo (ruta o URL)

Ejemplo:

```
export const defaultTheme = {  
  name: "default",
```

```
colors: {
  primary: "#000000",
  secondary: "#666666",
  background: "#ffffff",
  text: "#111111"
},

typography: {
  fontSans: "Inter",
  fontHeading: "Inter"
},

borderRadius: "8px",

branding: {
  storeName: "My Store",
  logo: "/logo.svg"
}
```

Cómo se usa el tema (concepto)

- **apps/web** lee el tema activo
- **apps/admin** puede cambiarlo
- **packages/ui** lo utiliza para estilizar componentes

1.4. Componentes reutilizables

1. Nivel 1: componentes básicos (obligatorios)

Estos son los más importantes porque todo depende de ellos:

- Button
- Input
- Label
- Card
- Badge

2. Nivel 2: layout UI

Componentes estructurales:

- Container
- Grid
- Stack (vertical/horizontal spacing)
- Section

3. Nivel 3: componentes de ecommerce

Aquí ya entras en dominio del negocio:

- ProductCard
- ProductGrid
- Price
- AddToCartButton
- QuantitySelector

4. Nivel 4: navegación

- Navbar
 - Footer
 - Breadcrumbs
 - Sidebar (para admin)
-

1.5. Documentación técnica inicial.

1. architecture.md

Qué contiene:

- visión general del sistema
- apps y responsabilidades
- packages y responsabilidades
- flujo del sistema (admin → api → web)

2. decisions.md (ADR ligero)

Qué contiene:

decisiones importantes del proyecto

- por qué se eligió cada cosa

Ejemplo de contenido:

- elección de Astro
- elección de Drizzle
- uso de monorepo

3. structure.md

Qué contiene:

- explicación del árbol de carpetas
- qué va en cada carpeta
- reglas de uso del repo

4. conventions.md

Qué contiene:

- naming conventions
 - reglas de código
 - estructura de componentes
 - reglas de imports entre packages/apps
-

FASE 2: Configuración IA

- Crear prompts para:
 - Arquitecto
 - Frontend
 - Backend
 - QA
 - Documentación
- Configurar flujo entre agentes.

FASE 3: Diseño UI

- Sistema de diseño.
- Componentes base.
- Layouts.
- Responsive.

FASE 4: Funcionalidades MVP

- Inicio
- Catálogo
- Producto
- Búsqueda
- Carrito
- Checkout
- Login
- Perfil
- Panel básico

Fase 5. Backend

- API
- Base de datos
- Autenticación
- Pedidos
- Inventario

Fase 6. Calidad

- Tests

- Lint
- Documentación
- Optimización

Fase 7. Template

- Temas
- Configuración
- Datos de ejemplo
- Scripts de personalización

Fase 8. Adaptación a clientes

- Branding
- Productos
- Integraciones
- Despliegue

Roadmap recomendado

1. Arquitectura
2. UI
3. Backend
4. MVP
5. Testing
6. Template
7. Automatización con IA
8. Versiones para clientes

Entregables

- Template reutilizable
- Documentación
- Prompts para agentes
- Guías de despliegue
- Base para múltiples clientes